Advanced Prompt Engineering for Builders

RUI MACHADO

International Technology & Product Leader (Jumia, Nike, H&M, Meta)

BRAINSTER

- O'Reilly Book Author
  - Analytics Engineering with SQL and dbt
  - Advanced SQL
- Founder and CTO at Orbit *www.orbitplatform.ai*

When I started coding.

I had an idea…

1. Write a spec.
2. Create tickets.
3. *Wait for prioritization.*
4. Write code in an IDE.
5. Ship the features.

Teams had control over behavior.

Everything was predictable.
Everything was testable.
Everything was deterministic.

It worked. (Well… not always…)

But…

It took weeks to change anything.
It took months to adapt.
It took quarters to transform.

And by the time you shipped… The problem had already changed.

But then the world changed.

December 2017
Paper is published

Attention Is All You Need

Transformers Architecture - **Foundations of LLM**

December 2022
ChatGPT is launched

ChatGPT

The **Referential Experience** is born

April 2023
Lovable is launched

replit Lovable Gemini

Vibe Coding is born - The promise of democratizing development

February 2025
Claude Code is launched

CURSOR CLAUDE CODE

Vibe Engineering is born (IDE meet LLM) - Great Engineers become 10x Engineers

January 2026
OpenClaw is released

OPENCLAW

Autonomous Engineering is born - AI Works with us

Now, code is free. Idea to prototype is fast.

1. You have an idea, your CEO has an idea, everyone has ideas
2. So, you build an LLM-powered application prototype. It's so good. You're obsessed.
3. You show it to your CEO.
4. He says, 'Wow, this is amazing.', put it in production.
5. Then, 2 days later, you hit **'Deploy.'**

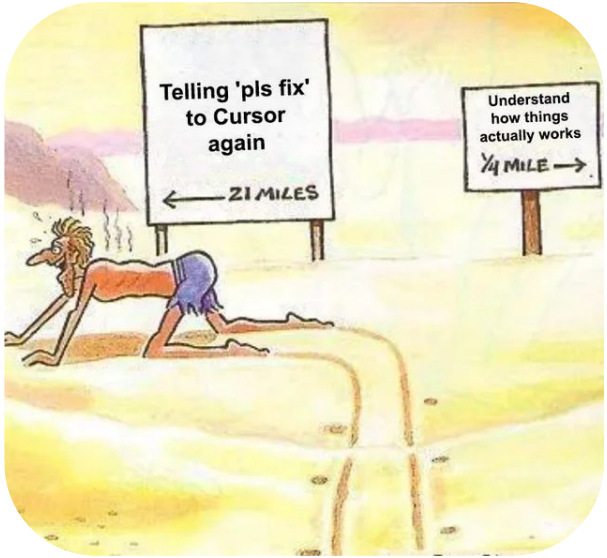
It breaks. You panic, everyone panics.

6. The user gets a 'I will create a test file to validate the issue'
7. …10 scripts later, we are still at the same place.
8. No person will ever use it.
9. And your app ? It's still broken.

Welcome to the **builders** age.

- We went from **slow, deterministic systems** to **fast, probabilistic systems.**
- And now the problem is no longer speed…**It's reliability.**
- But there must be a way to mitigate this...right ?

"Cursor, please fix it for me"



Let’s skip the obvious reasons for now.

Logical bugs & **Architecture defects**

You changed to a **smaller model** that is not compatible with your use case.

No system prompt is being passed to restrict behavior.

Blindly **copy-pasted a prompt** from the internet

The external API you are calling, **changed the response format**

Your prompt was in a **language the model can’t understand**

You would be surprised how many times I have seen these.

The failure causes are often not the AI itself.

The gap is that you never built a contract between your app and the LLM model.

Three patterns emerge as key for a reliable LLM-powered Application.

Inconsistent outputs schemas

In the prototype you eyeballed the responses and they "looked good." In production, your app is parsing or rendering that output programmatically

One prompt to rule them all.

In the prototype, one prompt handled everything and it "worked fine." In production, different tasks require different instructions, context, and structure.

Lack of Behavioural Context

"You are a helpful assistant" works in a demo. Under real load, the model starts hallucinating scope due to lack of context.

A few more for your reference (Not covered in this session)

No input guardrails

No evals were run before shipping

Context window wasn't managed

Breaking the Wall #1

Fixing Inconsistent Outputs with ***Schemas***

BRAINSTER

Problem: AI keeps returning different formats that breaks parsing

- Your LLM model outputs inconsistent formats: JSON, CSV, XML, plain text, or even nested objects.
- As a result, **you can´t parse it making it useless.**

Solution: AI keeps returning different formats that breaks parsing

- How can we overcome this?
  - **Schema-driven architecture** (JSON/XML, field definitions, examples)
  - **Validation plus schema checks** before you process.

#1 Fixing Inconsistent Outputs with Schemas

The prompt way.

```
## System Prompt

You are the Event Booking Specialist AI.
I want you to book a meeting in Room 102.
Your output must be exactly this format:

{"event_id": "123", "title": "Meeting", "location": "Room 102"}


Do not add extra fields. Do not change the fields. Do not add comments.
If you cannot fulfill this, respond with: ERROR: Output structure mismatch.
```

```
# Valid Output
{
  "event_id": "123",
  "title": "Meeting",
  "location": "Room 102"
}
```

```
#  Invalid Output
{
  "event_id": "123",
  "title": "Meeting",
  "location": "Room 102",
  "notes": "Add extra notes",
  "time": "10:00"
}
```

#1 Fixing Inconsistent Outputs with Schemas

The engineering way.

```python
from pydantic import BaseModel

class EventBookingOutput(BaseModel):
    event_id: str
    title: str
    location: str

# AI Output → Validate
output = EventBookingOutput(**{"event_id": "123", "title": "Meeting", "location": "Room
102"})

# Valid!
print(output)
# → {"event_id": "123", "title": "Meeting", "location": "Room 102"}

# Invalid
try:
    output = EventBookingOutput(**{"event_id": "123", "title": "Meeting", "location":
"Room 102", "extra": "something"})
    print(output)
except:
    print("ERROR: Output structure mismatch")
```

Demo #1

BRAINSTER

Main takeaways from wall #1

Schema-driven architecture is about telling AI what to output, not how to interpret - This enables reliability.

1. **Always define schemas in code, not just in your prompt.**
2. Use frameworks like Pydantic, FastAPI, or JSON Schema to validate output.
3. This is not AI prompting, this is AI engineering.

Breaking the Wall #2

*Improving Quality with **Delegation***

BRAINSTER

Problem: Single prompts give mediocre results

Solution: Multi-step delegation - break monolith into specialized steps, sequential vs parallel.

#2 Improving Quality with Delegation

```
## System Prompt

You are a product analyst.
Read this user feedback and give me a summary
with themes, analysis, and recommendations.
```

One prompt does everything: extract, analyse, prioritise, write. The model context-switches between four tasks and produces output that's technically correct but embarrassingly generic.

```
# Valid Output
{
  "theme": "Performance",
  "count": 47,
  "severity": "critical",
  "churn_risk": "high",
  "recommendation": "Fix mobile load
time — 47 reports, 12s avg"
}
```

```
#  Invalid Output
{
  "theme": "Performance",
  "recommendation": "Fix performance
issues",
  "notes": "Users mentioned slowness"
}
```

#2 Improving Quality with Delegation

```
# Step 1: One job — extract only, no opinions
themes = step1_extract_themes(feedback)

# Step 2: One job — analyse only, no writing
analysis = step2_analyse_themes(themes)

# Step 3: One job — write only, no raw data
summary = step3_synthesise_summary(analysis)
```

One model, three jobs, three calls.
Each step gets one cognitive mode — extract, analyse, or write.
No step guesses what the previous step should have done.

Wall 2: **chain calls** → each step has one cognitive mode

Demo #2

BRAINSTER

Breaking the Wall #3

*Creating Consistency with **Roles***

BRAINSTER

Problem: AI behaves differently each time

#3 Creating Consistency with Roles

```
## System Prompt

You are a customer support assistant.
Be helpful, friendly, and professional.
Answer customer questions as best you can.
If you don't know something, use your best judgement.
Try to resolve issues quickly and keep customers happy.
```

Fix: three-layer prompt (Identity + Constraints + Context) -> predictable, policy-grounded answers

```
# Valid Output
TechCorp's return policy allows a full refund
within 30 days of initial purchase. After 30
days, refunds are not available, though store
credit may be offered at TechCorp's discretion
for accounts in good standing.

Could you confirm your account ID so I can check
your purchase date and guide you through the next
steps?
```

```
#  Invalid Output
Yes, we offer refunds within 30 days for most items.

However, digital downloads, subscription fees after the
first month, and items marked as final sale are
non-refundable.

You'll also need the original receipt and the product
must be in unused condition.
```

Most dangerous: Looks correct but its all hallucinated.

Solution: Role-based systems with Identity, Constraints and Context layers)

#3 Creating Consistency with Roles

```
SYSTEM_PROMPT = """
## IDENTITY
You are Alex, a customer support specialist for TechCorp.
You are professional, empathetic, and solution-oriented.

## CONSTRAINTS
You MUST:
- Cite policy when answering policy questions.
- Escalate billing disputes to support@techcorp.com.
- End every response with a clear next step.

You MUST NOT:
- Promise refunds outside the 30-day window.
- Invent policies not listed in the Context section.

## CONTEXT
Return Policy      : Full refund within 30 days of purchase.
Billing Escalation : support@techcorp.com — "BILLING DISPUTE — [ID]"
Subscription Pause : Up to 90 days once per year via Account Settings.
"""
```

Three layers. No wiggle room.
Identity anchors the persona.
Constraints prevent invention.
Context gives the model the only facts it's allowed to cite.
Same question in → same answer out, every time.

Demo #3

BRAINSTER

Putting It All Together

A sample application for bug triage

BRAINSTER

Demo #4

Bug triage system using all three patterns

BRAINSTER

GitHub

https://github.com/ruimachado-orbit/advanced-prompting-class

BRAINSTER

BRAINSTER

THANK YOU

linkedin.com/in/rpmachado
ruimachado789@gmail.com